

DANIC GAST CORE BY Mr jammeh

```
import hashlib
import plyvel
import json
import time
import threading
import logging
import zmq
import socket
from flask import Flask, request, jsonify
from web3 import Web3
from eth_account import Account
import rlp
from eth_account._utils.legacy_transactions import Transaction
from typing import Dict, Any, List, Optional, Tuple

# =====
# ## 🚀 LOGGING SETUP
# =====
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger("DANIC_VALIDATOR")

# =====
# --- USER CONFIGURATION (Action Required: Replace Placeholder) ---
# =====
GENESIS_WALLET_ADDRESS = "0x0000000000000000000000000000000000000000"
INITIAL_BALANCE = 1000000000000000000000000

# *** ACTION REQUIRED: Replace placeholder with the Public IP of the ONLY other node ***
SINGLE_PEER_IP = "YOUR_SINGLE_PEER_PUBLIC_IP"

# NOTE: This list is used for ZMQ PUSH/CONNECT setup
KNOWN_PEER_IP_ADDRESSES = [SINGLE_PEER_IP]

# =====
# --- ARCHITECTURE CONSTANTS ---
# =====
GENESIS_HASH = "000000000001D4N1CG4ST"
H3_DOMAIN_PREFIX = "DANIC-H3-STATE-LOCK"
FEE_RATE = 0.01
RETRY_TIMEOUT_SECONDS = 1.0 # Time to wait before retrying a dependent package

# VBlock Synchronization Optimization
SYNC_CHUNK_SIZE = 35 # Number of VBlocks to request in a single batch (31.5 MB)

# ZMQ Ports
STATE_PORT = 6000 # For general account state query (REP/REQ - ZMQ)
P2P_PORT = 9000 # For GAST Transaction Gossip (PULL/PUSH - ZMQ)
H1_PUB_PORT = 9001 # For fast, asynchronous H1 state updates (PULL/PUSH - ZMQ)
VBLOCK_TCP_PORT = 7000 # For VBlock History requests (Sync - RAW TCP)

# VBlock Generator Triggers and State Keys
VBLOCK_TIME_LIMIT = 20.0 # 20 seconds interval for VBlock creation
VBLOCK_UPDATE_LIMIT = 3000 # <<< UPDATED: Max number of accounts in the buffer before trigger (approx 1MB VBlock)
VBLOCK_PREFIX = "vblock"
VBLOCK_STATE_KEY = "latest_vblock_num"
```

```
GENESIS_WALLET = Web3.to_checksum_address(GENESIS_WALLET_ADDRESS)
```

```
app = Flask(__name__)
```

```
db = plyvel.DB('danic_db', create_if_missing=True)
```

```
zmq_context = zmq.Context()
```

```
p2p_broadcaster_socket: Optional[zmq.Socket] = None
```

```
h1_broadcaster_socket: Optional[zmq.Socket] = None
```

```
last_vblock_time = time.time()
```

```
# =====
```

```
# ## 🗝 ACCOUNT LOCKING STRUCTURE
```

```
# =====
```

```
account_locks: Dict[str, threading.Lock] = {}
```

```
vblock_update_buffer: Dict[str, Dict[str, Any]] = {}
```

```
vblock_buffer_lock = threading.Lock()
```

```
def get_account_lock(addr: str) -> threading.Lock:
```

```
    addr = Web3.to_checksum_address(addr)
```

```
    if addr not in account_locks:
```

```
        account_locks[addr] = threading.Lock()
```

```
    return account_locks[addr]
```

```
# =====
```

```
# ## 🛠 UTILITY & DATABASE FUNCTIONS
```

```
# =====
```

```
def decode_raw_transaction(raw_tx: str) -> Optional[Dict[str, Any]]:
```

```
    try:
```

```
        if raw_tx.startswith('0x'): tx_bytes = bytes.fromhex(raw_tx[2:])
```

```
        else: tx_bytes = bytes.fromhex(raw_tx)
```

```
        decoded = rlp.decode(tx_bytes, Transaction).as_dict()
```

```
        sender = Account.recover_transaction(raw_tx)
```

```
        to = Web3.to_checksum_address(decoded['to']) if decoded.get('to') else None
```

```
        value = decoded['value']
```

```
        eth_tx_hash = Web3.keccak(tx_bytes).hex()
```

```
        return {'nonce': decoded['nonce'], "gasPrice": decoded['gasPrice'], "gas": decoded['gas'], "to": to, "value": value, "from": sender, "eth_tx_hash":
```

```
eth_tx_hash}
```

```
    except Exception as e:
```

```
        logger.error(f"[Decode Error] Failed to decode raw transaction: {e}")
```

```
    return None
```

```
def get_db_key(prefix: str, account_addr: str = "", key: str = "") -> bytes:
```

```
    if account_addr and key: return f"{prefix}:{Web3.to_checksum_address(account_addr)}:{key}".encode()
```

```
    if account_addr: return f"{prefix}:{Web3.to_checksum_address(account_addr)}".encode()
```

```
    return f"{prefix}:{key}".encode()
```

```
def get_latest_h1(addr: str) -> str:
```

```
    raw = db.get(get_db_key("h1", addr))
```

```
    return raw.decode() if raw else "0"*64
```

```
def get_all_active_accounts() -> List[str]:
```

```
    active_accounts = set()
```

```
    h1_prefix = "h1:".encode()
```

```
    for key, _ in db.iterator(prefix=h1_prefix):
```

```
        try:
```

```
            parts = key.decode().split(':', 1)
```

```
            if len(parts) == 2: active_accounts.add(parts[1])
```

```
        except Exception as e:
```

```
            logger.warning(f"Error parsing H1 key: {e}")
```

```
        continue
```

```

    return list(active_accounts)

def set_latest_h1(addr: str, h1_hash: str):
    db.put(get_db_key("h1", addr), h1_hash.encode())

def get_sequence(addr: str) -> int:
    raw = db.get(get_db_key("seq", addr))
    return int(raw.decode()) if raw else 0

def set_sequence(addr: str, seq: int):
    db.put(get_db_key("seq", addr), str(seq).encode())

def get_utxos(addr: str) -> Dict[str, int]:
    addr = Web3.to_checksum_address(addr)
    utxos = {}
    prefix = f"utxo:{addr}:".encode()
    for key, val in db.iterator(prefix=prefix):
        h4_key = key.decode().split(':')[1]
        utxos[h4_key] = int(val.decode())
    return utxos

def get_total_balance(addr: str) -> int:
    return sum(get_utxos(addr).values())

def index_gast_package(danic_tx_hash: str, eth_tx_hash: str, gast_package: Dict[str, Any], block_data: Dict[str, Any]):
    indexed_data = {
        **gast_package, "blockNumber": block_data["blockNumber"],
        "blockHash": block_data["blockHash"], "transactionIndex": block_data["transactionIndex"],
    }
    db.put(get_db_key("tx_danic", key=danic_tx_hash), json.dumps(indexed_data).encode())
    db.put(get_db_key("tx_eth", key=eth_tx_hash), json.dumps(indexed_data).encode())

def get_latest_vblock_number() -> int:
    """Retrieves the number of the last committed VBlock."""
    raw = db.get(get_db_key(VBLOCK_PREFIX, key=VBLOCK_STATE_KEY))
    return int(raw.decode()) if raw else 0

def get_vblock_by_number(num: int) -> Optional[Dict[str, Any]]:
    """Retrieves a complete VBlock record from the database."""
    raw = db.get(get_db_key(VBLOCK_PREFIX, key=str(num)))
    return json.loads(raw.decode()) if raw else None

def deserialize_h2_snapshot(snapshot_str: str) -> Dict[str, int]:
    """Helper to deserialize the H2 snapshot string into UTXO dictionary."""
    utxos = {}
    if not snapshot_str: return utxos
    for item in snapshot_str.split('|'):
        if ':' in item:
            key, value_str = item.split(':', 1)
            try: utxos[key] = int(value_str)
            except ValueError: logger.error(f"Invalid UTXO value in snapshot: {value_str}")
    return utxos

# =====
# ## 1 23 4 CORE HASH, VBLOCK COMMITMENT, AND VALIDATION FUNCTIONS
# =====

def canonical_serialize_h2_snapshot(utxos: Dict[str, int]) -> str:
    sorted_keys = sorted(utxos.keys())
    snapshot_parts = [f"{key}:{utxos[key]}" for key in sorted_keys]

```

```

return "|".join(snapshot_parts)

def compute_h3_expected(sender_id: str, prev_h1: str, tx_hash: str, sequence: int, sender_utxos: Dict[str, int]) -> str:
    # H3 formula: HASH(Domain | Prev H1 | Genesis | Sender ID | Sequence | H2 Snapshot | Tx Hash)
    canonical_h2_snapshot = canonical_serialize_h2_snapshot(sender_utxos)
    input_string = (
        H3_DOMAIN_PREFIX + "|" + prev_h1 + "|" + GENESIS_HASH + "|" +
        sender_id + "|" + str(sequence) + "|" + canonical_h2_snapshot + "|" + tx_hash
    )
    return hashlib.sha256(input_string.encode('utf-8')).hexdigest()

def compute_h4_key(tx_hash: str, output_index: str, output_value: int) -> str:
    input_string = (tx_hash + "|" + output_index + "|" + str(output_value))
    return hashlib.sha256(input_string.encode('utf-8')).hexdigest()

def compute_h1_hash(package_data: Dict[str, Any]) -> str:
    package_to_hash = package_data.copy()
    package_to_hash.pop('proposed_h3', None)
    serialized_package = json.dumps(package_to_hash, sort_keys=True).encode('utf-8')
    return hashlib.sha256(serialized_package).hexdigest()

def compute_vblock_h4(vblock_content: Dict[str, Any]) -> str:
    """VBlock Integrity Hash. Calculates the H4 hash over the entire VBlock content."""
    vblock_to_hash = vblock_content.copy()
    vblock_to_hash['state_updates'] = dict(sorted(vblock_to_hash['state_updates'].items()))
    serialized_content = json.dumps(vblock_to_hash, sort_keys=True).encode('utf-8')

    return hashlib.sha256(serialized_content).hexdigest()

def commit_vblock(vblock: Dict[str, Any]) -> bool:
    """Atomically applies all state changes specified in the VBlock to the LevelDB."""
    try:
        vblock_number = vblock['vblock_number']
        vblock_h4 = compute_vblock_h4(vblock)

        wb = db.write_batch()

        # 1. Update VBlock History and Chain Linkage
        wb.put(get_db_key(VBLOCK_PREFIX, key=str(vblock_number)), json.dumps(vblock).encode())

        # 2. Apply Overwrite Logic for all updated accounts
        for addr, state_data in vblock['state_updates'].items():
            addr = Web3.to_checksum_address(addr)

            # --- Deletion: Delete ALL existing UTXOs for the account (Overwrite Principle) ---
            prefix = f"utxo:{addr}:".encode()
            for key, _ in db.iterator(prefix=prefix):
                wb.delete(key)

            # --- Creation: Write new UTXOs from H2 Snapshot ---
            new_utxos = deserialize_h2_snapshot(state_data['New H2_Snapshot'])
            for h4_key, value in new_utxos.items():
                wb.put(get_db_key("utxo", addr, h4_key), str(value).encode())

            # --- Lineage Update: Update H1 and Sequence ---
            wb.put(get_db_key("h1", addr), state_data['New H1'].encode())
            wb.put(get_db_key("seq", addr), str(state_data['New Sequence']).encode())

```

```

# 3. Finalize VBlock Height
wb.put(get_db_key(VBLOCK_PREFIX, key=VBLOCK_STATE_KEY), str(vblock_number).encode())

wb.write()

    logger.info(f"✅ [VBLOCK COMMIT] VBlock #{vblock_number} committed (H4: {vblock_h4[:10]}...). {len(vblock['state_updates'])} accounts
updated.")
    return True
except Exception as e:
    logger.error(f"[VBLOCK COMMIT Error] Failed to commit VBlock #{vblock_number}: {e}", exc_info=True)
    return False

def process_gast_package(gast_package: Dict[str, Any], eth_tx_hash: str) -> Tuple[bool, str]:
    try:
        sender_id = Web3.to_checksum_address(gast_package.get('sender_id'))
        receiver_id = Web3.to_checksum_address(gast_package.get('receiver_id'))
        amount_sent = int(gast_package.get('amount', 0))
        tx_hash = gast_package.get('tx_hash')
        proposed_h3 = gast_package.get('proposed_h3')
        previous_h1 = gast_package.get('previous_h1_hash')
        input_utxo_keys = gast_package.get('input_utxo_keys', [])

        local_h1 = get_latest_h1(sender_id)
        current_sequence = get_sequence(sender_id)
        sender_utxos = get_utxos(sender_id)

        # --- L1: H1 VALIDATION (Lineage Check) ---
        if previous_h1 != local_h1:
            return False, f"Error L1: H1 mismatch. Expected: {local_h1[:10]}... Got: {previous_h1[:10]}..."

        # --- L3: H3 PRE-CHECK (State Integrity Lock) ---
        expected_h3 = compute_h3_expected(sender_id, local_h1, tx_hash, current_sequence, sender_utxos)
        if expected_h3 != proposed_h3:
            return False, "Error L3: H3 mismatch. Potential double-spend/divergence."

        # --- L2: H2 VALIDATION (Viability Check) ---
        total_input_value = sum(sender_utxos[key] for key in input_utxo_keys if key in sender_utxos)
        consumed_utxos = {key: sender_utxos[key] for key in input_utxo_keys if key in sender_utxos}
        if len(consumed_utxos) != len(input_utxo_keys):
            return False, "Error L2: One or more referenced UTXO keys not found."

        platform_fee = int(amount_sent * FEE_RATE)
        required_amount = amount_sent + platform_fee
        if total_input_value < required_amount:
            return False, f"Error L2: Insufficient funds. Required: {required_amount}, Available: {total_input_value}."
        change_amount = total_input_value - required_amount

        # --- CALCULATE NEW STATE (Deferred to VBlock Commit) ---
        new_utxos_sender: Dict[str, int] = {}
        if change_amount > 0:
            h4_change_key = compute_h4_key(tx_hash, "CHG", change_amount)
            new_utxos_sender[h4_change_key] = change_amount
        h4_receiver_key = compute_h4_key(tx_hash, "RCV", amount_sent)
        new_utxos_receiver: Dict[str, int] = {h4_receiver_key: amount_sent}
        h4_fee_key = compute_h4_key(tx_hash, "FEE", platform_fee)
        new_utxos_fee: Dict[str, int] = {h4_fee_key: platform_fee}

```

```

# Combine old UTXOs and new UTXOs for the sender's H2 calculation
sender_post_utxos = {k: v for k, v in sender_utxos.items() if k not in consumed_utxos}
sender_post_utxos.update(new_utxos_sender)

# Calculate the new H1 hash and sequence
next_sequence = current_sequence + 1
gast_package["sequence"] = next_sequence # Update package for H1 calculation
new_h1 = compute_h1_hash(gast_package)

# --- QUEUE THE FINALIZED STATE FOR VBlock COMMITMENT ---
with vblock_buffer_lock:
    # 1. Sender State
    vblock_update_buffer[sender_id] = {
        "New H1": new_h1,
        "New Sequence": next_sequence,
        "New H2_Snapshot": canonical_serialize_h2_snapshot(sender_post_utxos),
        "Source_Tx": tx_hash
    }
    # 2. Receiver State
    receiver_utxos_pre = get_utxos(receiver_id)
    receiver_utxos_pre.update(new_utxos_receiver)
    vblock_update_buffer[receiver_id] = {
        "New H1": get_latest_h1(receiver_id),
        "New Sequence": get_sequence(receiver_id),
        "New H2_Snapshot": canonical_serialize_h2_snapshot(receiver_utxos_pre),
        "Source_Tx": tx_hash
    }
    # 3. Genesis Fee State
    genesis_utxos_pre = get_utxos(GENESIS_WALLET)
    genesis_utxos_pre.update(new_utxos_fee)
    vblock_update_buffer[GENESIS_WALLET] = {
        "New H1": get_latest_h1(GENESIS_WALLET),
        "New Sequence": get_sequence(GENESIS_WALLET),
        "New H2_Snapshot": canonical_serialize_h2_snapshot(genesis_utxos_pre),
        "Source_Tx": tx_hash
    }

# --- OPTIMISTIC LINEAGE FIX & GOSSIP ---
set_sequence(sender_id, next_sequence)
set_latest_h1(sender_id, new_h1)
threading.Thread(target=broadcast_h1_update, args=(sender_id, new_h1, next_sequence)).start()

# Index the transaction against the *next* VBlock number
virtual_block_hash = '0x' + tx_hash
virtual_block_number = "0x" + hex(get_latest_vblock_number() + 1)[2:]
block_data = {"blockNumber": virtual_block_number, "blockHash": virtual_block_hash, "transactionIndex": "0x0"}
index_gast_package(tx_hash, eth_tx_hash, gast_package, block_data)

return True, tx_hash

except Exception as e:
    logger.error(f"System Error during validation: {e}", exc_info=True)
    return False, f"Server error: {e}"

# =====
# ## 🔄 RAW TCP SYNCHRONIZATION CLIENT LOGIC (Port 7000)

```

```

# =====

def request_peer_data_tcp(peer_ip: str, request_data: Dict[str, Any]) -> Optional[Dict[str, Any]]:
    """
    Helper to send a synchronous request to a peer's VBlock History Server (7000)
    using Raw TCP sockets with length prefix framing.
    """
    port = VBLOCK_TCP_PORT

    # 1. Prepare message with 4-byte length header
    request_json = json.dumps(request_data)
    message = len(request_json).to_bytes(4, byteorder='big') + request_json.encode('utf-8')

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(5.0) # 5 second total timeout

    try:
        # Crucial delay (1s) to allow the peer's bind operation to stabilize
        time.sleep(1.0)

        s.connect((peer_ip, port))

        # 2. Send the full message
        s.sendall(message)

        # 3. Receive the 4-byte response length header
        response_len_bytes = b''
        while len(response_len_bytes) < 4:
            chunk = s.recv(4 - len(response_len_bytes))
            if not chunk: raise ConnectionError("Peer closed connection during header receive.")
            response_len_bytes += chunk

        response_len = int.from_bytes(response_len_bytes, byteorder='big')

        # 4. Receive the full JSON payload
        response_data = b''
        while len(response_data) < response_len:
            chunk = s.recv(response_len - len(response_data))
            if not chunk: raise ConnectionError("Peer closed connection during payload receive.")
            response_data += chunk

        return json.loads(response_data.decode('utf-8'))

    except socket.timeout:
        logger.warning(f"[TCP SYNC] Peer {peer_ip}:{port} connection/read timeout.")
        return None
    except ConnectionRefusedError:
        logger.warning(f"[TCP SYNC] Peer {peer_ip}:{port} actively refused connection (Server not running or port closed).")
        return None
    except ConnectionError as e:
        logger.warning(f"[TCP SYNC] Connection error with {peer_ip}:{port}: {e}")

```

```

        return None
    except Exception as e:
        logger.error(f"[TCP SYNC] Unexpected error during request: {e}", exc_info=True)
        return None
    finally:
        s.close()

def synchronize_from_peer() -> bool:
    """
    Attempts to connect to the SINGLE_PEER_IP to download VBlocks
    in chunks and achieve network synchronization using Raw TCP.
    """
    peer_ip = SINGLE_PEER_IP

    if peer_ip == "YOUR_SINGLE_PEER_PUBLIC_IP" or not peer_ip:
        logger.error("[SYNC] SINGLE_PEER_IP is not configured. Cannot attempt sync.")
        return False

    logger.info(f"[SYNC] Trying to connect to single peer: {peer_ip}")

    # 1. Request Latest VBlock Number
    response = request_peer_data_tcp(peer_ip, {"request_type": "GET_LATEST_VBLOCK_NUMBER"})

    if not response or response.get("status") != "success":
        logger.warning(f"[SYNC] Failed to get latest block from {peer_ip} (TCP error/no blocks). Falling back to local bootstrap.")
        return False

    latest_peer_vblock_num = response.get("latest_vblock_number", 0)
    local_vblock_num = get_latest_vblock_number()

    if latest_peer_vblock_num <= local_vblock_num:
        logger.info(f"[SYNC] Peer {peer_ip} state is not newer (Local: {local_vblock_num}, Peer: {latest_peer_vblock_num}). No sync needed.")
        return False

    current_block = local_vblock_num + 1

    # 2. Iterative VBlock Download and Commit (Chunked)
    while current_block <= latest_peer_vblock_num:
        # Calculate the batch range using the SYNC_CHUNK_SIZE
        start_block = current_block
        end_block = min(current_block + SYNC_CHUNK_SIZE - 1, latest_peer_vblock_num)

        logger.info(f"[SYNC] Requesting VBlock range #{start_block} to #{end_block} ({end_block - start_block + 1} blocks) from peer...")

        # New TCP Request for the Chunk (Range)
        vblock_resp = request_peer_data_tcp(peer_ip, {
            "request_type": "GET_VBLOCKS_BY_RANGE",
            "start_block_number": start_block,
            "end_block_number": end_block
        })

```



```

if not vblock_resp or vblock_resp.get("status") != "success":
    logger.error(f"[SYNC] Failed to download VBlock chunk {start_block}-{end_block}. Aborting sync.")
    return False

# Commit blocks in sequence
downloaded_vblocks = vblock_resp.get("vblocks", [])

# Critical: VBlocks must be committed in order to maintain parent_h4 lineage
for vblock in downloaded_vblocks:
    commit_vblock(vblock)

# Update pointer for the next batch
current_block = end_block + 1

logger.info("✅ **SYNC COMPLETE**: Node state successfully synchronized from peer.")
return True

# =====
# ## 🌐 ZMQ P2P GOSSIP & STATE SYNC (Port 9000, 9001, 6000)
# =====

def setup_p2p_broadcaster():
    """Initializes and connects the global ZMQ PUSH socket for GAST packages."""
    global p2p_broadcaster_socket
    if p2p_broadcaster_socket is None:
        p2p_broadcaster_socket = zmq_context.socket(zmq.PUSH)
        p2p_broadcaster_socket.setsockopt(zmq.LINGER, 0)
        p2p_broadcaster_socket.setsockopt(zmq.HEARTBEAT_IVL, 500)
        p2p_broadcaster_socket.setsockopt(zmq.HEARTBEAT_TIMEOUT, 1000)

        if SINGLE_PEER_IP != "YOUR_SINGLE_PEER_PUBLIC_IP":
            p2p_broadcaster_socket.connect(f"tcp://{SINGLE_PEER_IP}:{P2P_PORT}")

            logger.info(f"[GOSSIP Setup] Connected PUSH socket to {SINGLE_PEER_IP}:{P2P_PORT}")
        else:
            logger.warning("[GOSSIP Setup] Peer IP not configured. Skipping PUSH connection.")

def broadcast_gast_package(gast_package: Dict[str, Any]):
    global p2p_broadcaster_socket
    if p2p_broadcaster_socket is None:
        logger.error("[GOSSIP] Broadcaster socket not initialized.")
        return
    package_json = json.dumps(gast_package)
    try:
        p2p_broadcaster_socket.send_string(package_json, zmq.NOBLOCK)

        logger.info(f"[GOSSIP] Broadcasted TX {gast_package['tx_hash'][:10]}... to peer:")
    except zmq.error.Again:
        logger.warning(f"[GOSSIP] Failed to broadcast TX {gast_package['tx_hash'][:10]}... (Buffer full).")
    except Exception as e:
        logger.error(f"[GOSSIP] Failed to send package: {e}")

def setup_h1_broadcaster():
    """Initializes and connects the global ZMQ PUSH socket for H1 updates."""
    global h1_broadcaster_socket
    if h1_broadcaster_socket is None:

```

```

h1_broadcaster_socket = zmq_context.socket(zmq.PUSH)
h1_broadcaster_socket.setsockopt(zmq.LINGER, 0)

if SINGLE_PEER_IP != "YOUR_SINGLE_PEER_PUBLIC_IP":
    h1_broadcaster_socket.connect(f"tcp://{SINGLE_PEER_IP}:{H1_PUB_PORT}")

    logger.info(f"[H1 GOSSIP Setup] Connected PUSH socket for H1 to {SINGLE_PEER_IP}:{H1_PUB_PORT}")
else:
    logger.warning("[H1 GOSSIP Setup] Peer IP not configured. Skipping PUSH connection.")

def broadcast_h1_update(addr: str, h1_hash: str, sequence: int):
    global h1_broadcaster_socket
    if h1_broadcaster_socket is None: return
    update_data = json.dumps({"addr": addr, "h1_hash": h1_hash, "sequence": sequence})
    try:
        h1_broadcaster_socket.send_string(update_data, zmq.NOBLOCK)
        logger.debug(f"[H1 GOSSIP] Broadcasted H1 update for {addr[:10]}...")
    except zmq.error.Again:
        logger.warning(f"[H1 GOSSIP] Failed to broadcast H1 update (Buffer full).")
    except Exception as e:
        logger.error(f"[H1 GOSSIP] Failed to send H1 update: {e}")

def run_p2p_listener_zmq():
    s = zmq_context.socket(zmq.PULL)
    s.bind(f"tcp://0.0.0.0:{P2P_PORT}")

    logger.info(f"[P2P Listener] Transaction Gossip socket started on port {P2P_PORT}")

dependency_buffer: Dict[str, Tuple[Dict[str, Any], str, float]] = {}

while True:
    try:
        package_json = s.recv_string()
        gast_package = json.loads(package_json)
        tx_hash = gast_package.get('tx_hash', 'UNKNOWN')
        eth_tx_hash = gast_package.get('eth_tx_hash')
        sender_id = gast_package.get('sender_id')

        logger.info(f"[P2P GOSSIP] Received package {tx_hash[:10]}... from peer. Starting verification.")

        if not eth_tx_hash or not sender_id:
            logger.warning(f"[P2P GOSSIP] Package {tx_hash[:10]}... rejected: Malformed package.")
            continue

        account_lock = get_account_lock(sender_id)

        def attempt_processing(pkg, eth_hash):
            with account_lock:
                return process_gast_package(pkg, eth_hash)

        success, result = attempt_processing(gast_package, eth_tx_hash)

        if not success and ("Error L1" in result or "Error L3" in result):
            logger.warning(f"[P2P GOSSIP] Package {tx_hash[:10]}... failed (Code {result[7:9]}). Buffering and retrying in {RETRY_TIMEOUT_SECONDS}s.")
            dependency_buffer[tx_hash] = (gast_package, eth_tx_hash, time.time() + RETRY_TIMEOUT_SECONDS)

```

```

elif success:
    logger.info(f"[P2P GOSSIP] Package {result[:10]}... verified and state queued locally.")

else:
    logger.warning(f"[P2P GOSSIP] Package {tx_hash[:10]}... rejected: {result}")

keys_to_remove = []
for d_tx_hash, (d_pkg, d_eth_hash, retry_time) in dependency_buffer.items():
    if time.time() >= retry_time:
        d_sender_id = d_pkg.get('sender_id')
        d_account_lock = get_account_lock(d_sender_id)

        with d_account_lock:
            d_success, d_result = process_gast_package(d_pkg, d_eth_hash)

        if d_success:
            logger.info(f"[P2P GOSSIP/RETRY] Package {d_tx_hash[:10]}... successful on retry.")

            keys_to_remove.append(d_tx_hash)
        else:
            logger.warning(f"[P2P GOSSIP/RETRY] Package {d_tx_hash[:10]}... failed permanently: {d_result}")
            keys_to_remove.append(d_tx_hash)

for key in keys_to_remove:
    del dependency_buffer[key]

except Exception as e:
    logger.error(f"[P2P Listener Error] {e}", exc_info=True)

def run_h1_listener_zmq():
    s = zmq_context.socket(zmq.PULL)
    s.bind(f"tcp://0.0.0.0:{H1_PUB_PORT}")

    logger.info(f"[H1 Listener] H1 State Update socket started on port {H1_PUB_PORT}")

    while True:
        try:
            update_json = s.recv_string()
            update_data = json.loads(update_json)
            addr = update_data.get('addr')
            new_h1 = update_data.get('h1_hash')
            new_seq = update_data.get('sequence')

            if not addr or not new_h1 or new_seq is None: continue

            account_lock = get_account_lock(addr)
            with account_lock:
                current_seq = get_sequence(addr)
                if new_seq > current_seq:
                    set_latest_h1(addr, new_h1)
                    set_sequence(addr, new_seq)
                    logger.debug(f"[H1 GOSSIP] State updated for {addr[:10]}... to Seq:{new_seq}, H1:{new_h1[:10]}...")
                elif new_seq < current_seq:
                    logger.warning(f"[H1 GOSSIP] Received stale H1 update for {addr[:10]}... (Remote Seq: {new_seq}, Local Seq: {current_seq}).")

        except Exception as e:
            logger.error(f"[H1 Listener Error] {e}", exc_info=True)

```

```

def run_state_query_zmq():
    s = zmq_context.socket(zmq.REP)
    s.bind(f"tcp://0.0.0.0:{STATE_PORT}")

    logger.info(f"[ZMQ] State Query socket started on port {STATE_PORT}")

    while True:
        try:
            msg = s.recv_json()
            request_type = msg.get("request_type")

            if request_type == "GET_ALL_ACCOUNTS":
                accounts = get_all_active_accounts()
                s.send_json({"status": "success", "accounts": accounts})

                logger.info(f"[ZMQ] Served {len(accounts)} active account addresses for sync.")

                continue

            elif request_type == "GET_ACCOUNT_STATE":
                address = msg.get("address")
                if not address:
                    s.send_json({"status": "error", "message": "Missing address parameter."})
                    continue

                h1_hash = get_latest_h1(address)
                utxos = get_utxos(address)
                sequence = get_sequence(address)
                s.send_json({"status": "success", "address": address, "h1_hash": h1_hash, "utxos": utxos, "sequence": sequence})
                logger.debug(f"[ZMQ] Served state for account {address[:10]}...")
                continue

            s.send_json({"status": "error", "message": "Invalid request_type."})
        except Exception as e:
            logger.error(f"[ZMQ State Query Error] {e}", exc_info=True)
            try: s.send_json({"status": "error", "message": str(e)})
            except: pass

# =====
# ## 📡 RAW TCP VBLOCK SERVER LOGIC (Port 7000)
# =====

def handle_vblock_client(conn: socket.socket, addr: Tuple[str, int]):
    """Handles a single VBlock history request over the TCP connection."""
    logger.debug(f"Handling VBlock client connection from {addr[0]}")
    conn.settimeout(5.0)

def send_response(response_dict: Dict[str, Any]):
    """Helper to send the response with a 4-byte length header."""
    response_json = json.dumps(response_dict)
    response_message = len(response_json).to_bytes(4, byteorder='big') + response_json.encode('utf-8')
    conn.sendall(response_message)

try:
    # 1. Receive 4-byte request length header
    request_len_bytes = b""
    while len(request_len_bytes) < 4:
        chunk = conn.recv(4 - len(request_len_bytes))
        if not chunk: return

```

```

request_len_bytes += chunk

request_len = int.from_bytes(request_len_bytes, byteorder='big')

# 2. Receive the full JSON payload
request_data = b""
while len(request_data) < request_len:
    chunk = conn.recv(request_len - len(request_data))
    if not chunk: return
    request_data += chunk

msg = json.loads(request_data.decode('utf-8'))
request_type = msg.get("request_type")

# 3. Process Request
if request_type == "GET_VBLOCK_BY_NUMBER":
    vblock_num = msg.get("vblock_number")
    if not isinstance(vblock_num, int) or vblock_num <= 0:
        send_response({"status": "error", "message": "Invalid vblock_number parameter:"})
        return

    vblock = get_vblock_by_number(vblock_num)

    if vblock: send_response({"status": "success", "vblock": vblock})
    else: send_response({"status": "error", "message": f"VBlock #{vblock_num} not found."})

elif request_type == "GET_VBLOCKS_BY_RANGE":
    # New handler for chunked download
    start_num = msg.get("start_block_number")
    end_num = msg.get("end_block_number")

    if not isinstance(start_num, int) or not isinstance(end_num, int) or start_num <= 0 or start_num > end_num:
        send_response({"status": "error", "message": "Invalid block range parameters:"})
        return

    vblocks_list = []
    for num in range(start_num, end_num + 1):
        vblock = get_vblock_by_number(num)
        if vblock: vblocks_list.append(vblock)
        else:
            # Stop if a gap is found, return what we have so far
            logger.warning(f"[TCP SERVER] VBlock #{num} not found within requested range. Returning partial chunk.")
            break

    if vblocks_list:
        send_response({"status": "success", "vblocks": vblocks_list})
    else:
        send_response({"status": "error", "message": f"No VBlocks found in range {start_num}-{end_num}."})

elif request_type == "GET_LATEST_VBLOCK_NUMBER":
    latest_num = get_latest_vblock_number()
    send_response({"status": "success", "latest_vblock_number": latest_num})

```

```

else:
    send_response({"status": "error", "message": "Invalid request_type."})

except socket.timeout:
    logger.warning(f"[TCP SERVER] Client {addr[0]} timed out.")
except Exception as e:
    logger.error(f"[TCP SERVER Error] Handling client {addr[0]}: {e}", exc_info=True)
finally:
    conn.close()

def run_vblock_tcp_server():
    """Main TCP Server thread for VBlock History requests (Port 7000)."""
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(('0.0.0.0', VBLOCK_TCP_PORT))
    s.listen(5)

    logger.info(f"[TCP] VBlock History server started on port {VBLOCK_TCP_PORT}")

    while True:
        try:
            conn, addr = s.accept()
            threading.Thread(target=handle_vblock_client, args=(conn, addr)).start()
        except Exception as e:
            logger.error(f"[TCP SERVER] Main loop error: {e}", exc_info=True)
            time.sleep(1)

def run_vblock_generator():
    """VBlock Generator. Creates and commits VBlocks based on time or size limits."""
    global last_vblock_time

    while True:
        time.sleep(1)

        with vblock_buffer_lock:
            buffer_size = len(vblock_update_buffer)
            time_elapsed = time.time() - last_vblock_time

            should_create_vblock = (
                buffer_size >= VBLOCK_UPDATE_LIMIT or
                (time_elapsed >= VBLOCK_TIME_LIMIT and buffer_size > 0)
            )

            if should_create_vblock:
                logger.info(f"[VBlock Generator] Triggered (Size: {buffer_size}/{VBLOCK_UPDATE_LIMIT}, Time: {time_elapsed:.1f}s/{VBLOCK_TIME_LIMIT}s).")

                vblock_number = get_latest_vblock_number() + 1
                parent_vblock = get_vblock_by_number(vblock_number - 1)
                parent_h4 = compute_vblock_h4(parent_vblock) if parent_vblock else GENESIS_HASH

                new_vblock = {
                    "vblock_number": vblock_number,
                    "parent_h4": parent_h4,

```

```

        "timestamp": int(time.time()),
        "state_updates": vblock_update_buffer.copy()
    }

    success = commit_vblock(new_vblock)

    if success:
        vblock_update_buffer.clear()
        last_vblock_time = time.time()
    else:
        logger.error("[VBlock Generator] Commit failed. Retaining buffer for next attempt.")

    elif time_elapsed >= VBLOCK_TIME_LIMIT and buffer_size == 0:
        last_vblock_time = time.time()
        logger.debug(f"[VBlock Generator] Skipping VBlock commit: No updates in buffer.")

# =====
# ## ❌ INITIALIZATION
# =====

def init_node():
    """Checks DB status and orchestrates either Sync or Local Bootstrap."""
    initial_h1 = get_latest_h1(GENESIS_WALLET)

    if initial_h1 == "0"*64:
        logger.warning("[Init] Local database is empty (Cold Start detected).")

    # 1. ATTEMPT P2P SYNCHRONIZATION
    sync_success = synchronize_from_peer()

    if sync_success:
        pass
    else:
        # 2. FALLBACK TO LOCAL GENESIS BOOTSTRAP
        logger.warning("[Init] Synchronization failed or no valid peer was configured. Bootstrapping with local Genesis state.")

    # --- BOOTSTRAP VBLOCK GENESIS ---
    genesis_tx_hash = "0"*64
    genesis_h4_key = compute_h4_key(genesis_tx_hash, "GENESIS", INITIAL_BALANCE)
    genesis_utxos = {genesis_h4_key: INITIAL_BALANCE}

    genesis_vblock = {
        "vblock_number": 1,
        "parent_h4": GENESIS_HASH,
        "timestamp": int(time.time()),
        "state_updates": {
            GENESIS_WALLET: {
                "New H1": genesis_tx_hash,
                "New Sequence": 1,
                "New H2_Snapshot": canonical_serialize_h2_snapshot(genesis_utxos)
            }
        }
    }

```

```

    }
    local_bootstrap_success = commit_vblock(genesis_vblock)
    # --- END BOOTSTRAP VBLOCK GENESIS ---

    if local_bootstrap_success:
        logger.info(f"✅ **BOOTSTRAP COMPLETE**: Node initialized with local Genesis VBlock.")
    else:
        logger.error(f"❌ **FATAL ERROR**: Genesis state creation failed.")
    else:
        latest_vblock = get_latest_vblock_number()
        logger.info(f"📁 **EXISTING STATE LOADED**: Node state found (VBlock Height: {latest_vblock}, H1: {initial_h1[:10]}...). Skipping sync and ready
for transactions.")

# =====
# ### 🛡️ RPC METHODS (PROTECTED BY ACCOUNT LOCK)
# =====
@app.route('/', methods=['POST'])
def rpc():
    try:
        data = request.get_json()
        method = data.get('method')
        params = data.get('params', [])
        rpc_id = data.get('id', '1')

        if method == 'eth_blockNumber': return jsonify({"jsonrpc": "2.0", "id": rpc_id, "result": hex(get_latest_vblock_number())})
        elif method == 'eth_getBalance':
            balance = get_total_balance(params[0]) if params else 0
            return jsonify({"jsonrpc": "2.0", "id": rpc_id, "result": hex(balance)})
        elif method == 'eth_getTransactionCount':
            nonce = get_sequence(params[0]) if params else 0
            return jsonify({"jsonrpc": "2.0", "id": rpc_id, "result": hex(nonce)})

        elif method == 'eth_sendRawTransaction':
            raw_tx = params[0] if params else None
            if not raw_tx: return jsonify({"jsonrpc": "2.0", "id": rpc_id, "error": {"code": -32602, "message": "Missing raw transaction"}})
            tx_info = decode_raw_transaction(raw_tx)
            if not tx_info: return jsonify({"jsonrpc": "2.0", "id": rpc_id, "error": {"code": -32000, "message": "Invalid RLP transaction format"}})

            sender_id, receiver_id, amount_sent, eth_tx_hash = tx_info['from'], tx_info['to'], tx_info['value'], tx_info['eth_tx_hash']

            account_lock = get_account_lock(sender_id)
            with account_lock:

                available_utxos = get_utxos(sender_id)
                input_utxo_keys = list(available_utxos.keys())
                if not input_utxo_keys: return jsonify({"jsonrpc": "2.0", "id": rpc_id, "error": {"code": -32000, "message": "No available UTXOs for sender."}})

                canonical_tx_body = f"{{sender_id}}{{receiver_id}}{{amount_sent}}{{time.time()}}"
                danic_tx_hash = hashlib.sha256(canonical_tx_body.encode('utf-8')).hexdigest()

                previous_h1 = get_latest_h1(sender_id)
                current_sequence = get_sequence(sender_id)

```



```

proposed_h3_val = compute_h3_expected(
    sender_id,
    previous_h1,
    danic_tx_hash,
    current_sequence,
    available_utxos
)

gast_package = {
    "sender_id": sender_id, "receiver_id": receiver_id, "amount": amount_sent,
    "tx_hash": danic_tx_hash, "previous_h1_hash": previous_h1, "input_utxo_keys": input_utxo_keys,
    "proposed_h3": proposed_h3_val,
    "sequence": current_sequence + 1,
    "eth_tx_hash": eth_tx_hash
}

success, result_message = process_gast_package(gast_package, eth_tx_hash)

if success:
    logger.info(f"[RPC] Transaction {eth_tx_hash[:10]}... verified, queued for VBlock, and broadcasting.")
    threading.Thread(target=broadcast_gast_package, args=(gast_package,)).start()
    return jsonify({"jsonrpc": "2.0", "id": rpc_id, "result": eth_tx_hash})
else:
    logger.warning(f"[RPC] Transaction rejected: {result_message}")
    return jsonify({"jsonrpc": "2.0", "id": rpc_id, "error": {"code": -32001, "message": f"GAST Package Rejected: {result_message}"}})

logger.warning(f"[RPC] Unsupported method: {method}")
return jsonify({"jsonrpc": "2.0", "id": rpc_id, "error": {"code": -32601, "message": "Method not found or unsupported."}})

except Exception as e:
    logger.error(f"[RPC Error] {e}", exc_info=True)
    return jsonify({"jsonrpc": "2.0", "error": {"code": -32000, "message": "Server error"}})

# =====
# --- MAIN EXECUTION BLOCK ---
# =====

if __name__ == '__main__':
    init_node()

    setup_p2p_broadcaster()
    setup_h1_broadcaster()

    # Start all ZMQ communication threads (State Query, GAST Gossip, H1 Updates)
    threading.Thread(target=run_state_query_zmq, daemon=True).start()
    threading.Thread(target=run_p2p_listener_zmq, daemon=True).start()
    threading.Thread(target=run_h1_listener_zmq, daemon=True).start()

    # Start the RAW TCP VBlock History Server
    threading.Thread(target=run_vblock_tcp_server, daemon=True).start()

```

```

# Start the VBlock finality threads
threading.Thread(target=run_vblock_generator, daemon=True).start()

logger.info(f"\n--- BLOCKLESS VALIDATOR READY ---")

logger.info(f"1. External Interface (MetaMask): Listening on port 8041 (RPC)")

logger.info(f"2. Internal Interface (State Query): Listening on port {STATE_PORT} (ZMQ REQ/REP)")

logger.info(f"3. Internal Interface (P2P Gossip): Listening on port {P2P_PORT} (GAST PULL)")

logger.info(f"4. Internal Interface (H1 Gossip): Listening on port {H1_PUB_PORT} (H1 PULL)")

logger.info(f"5. Internal Interface (VBlock History): Listening on port {VBLOCK_TCP_PORT} (RAW TCP)")

logger.info(f"6. **VBlock Sync Chunk Size**: {SYNC_CHUNK_SIZE} blocks (Optimized for IBD speed).")

logger.info(f"7. **VBlock Update Limit**: {VBLOCK_UPDATE_LIMIT} account updates (High-throughput target).")

logger.info(f"8. **Single Peer Configured**: {SINGLE_PEER_IP}")

logger.info(f"-----\n")
app.run(host='0.0.0.0', port=8041, threaded=True)

```

UPDATE NEEDED TO BE THIER

```

import socket
import json
import hashlib
import plyvel
import struct
import logging

# =====
# CONFIG
# =====

HANDSHAKE_DOMAIN = b"DANIC-HANDSHAKE-V1"

logging.basicConfig(
    filename="danic_audit.log",
    level=logging.WARNING,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

# =====
# 1. ATOMIC STATE ENGINE (LEVELDB)
# =====

def apply_vblock_atomic(
    db,
    sender_id,
    consumed_h4s,
    new_outputs,
    new_h1,
    new_seq,
    peer_id=None
):
    """
    Final Danic atomic state transition.
    Crash-safe, fork-safe, auditable.

```

```

"""
addr_prefix = sender_id.lower().encode()

try:
    # -----
    # PRE-CHECKS
    # -----

    # Sequence monotonicity check
    seq_key = b"seq:" + addr_prefix
    current_seq = db.get(seq_key)
    current_seq = int(current_seq.decode()) if current_seq else 0

    if new_seq != current_seq + 1:
        raise Exception("Sequence violation")

    # UTXO existence check
    for h4 in consumed_h4s:
        utxo_key = b"utxo:" + addr_prefix + b":" + h4.encode()
        if db.get(utxo_key) is None:
            raise Exception(f"UTXO already spent: {h4}")

    # -----
    # ATOMIC COMMIT
    # -----
    with db.write_batch(transaction=True) as wb:
        # Spend inputs
        for h4 in consumed_h4s:
            wb.delete(b"utxo:" + addr_prefix + b":" + h4.encode())

        # Create outputs
        for h4_key, value in new_outputs.items():
            wb.put(
                b"utxo:" + addr_prefix + b":" + h4_key.encode(),
                str(value).encode()
            )

        # Lock lineage
        wb.put(b"h1:" + addr_prefix, new_h1.encode())
        wb.put(seq_key, str(new_seq).encode())

    return True

except Exception as e:
    # -----
    # AUDIT LOGGING
    # -----
    logging.warning(
        f"AUDIT_FAIL | peer={peer_id} | sender={sender_id} | "
        f"seq={new_seq} | reason={str(e)}"
    )
    return False, str(e)

# =====
# 2. HANDSHAKE SECURITY
# =====

def compute_handshake_hash(genesis_h4, block_num, block_h4):
    """
    Genesis + Height + Latest State commitment

```

```

"""
payload = (
    HANDSHAKE_DOMAIN +
    bytes.fromhex(genesis_h4) +
    block_num.to_bytes(8, "big") +
    bytes.fromhex(block_h4)
)
return hashlib.sha256(payload).hexdigest()

# =====
# 3. NETWORK HELPERS
# =====

def send_json(conn, data):
    raw = json.dumps(data).encode()
    conn.sendall(struct.pack(">I", len(raw)) + raw)

def recv_json(conn):
    header = conn.recv(4)
    if not header:
        return None
    size = struct.unpack(">I", header)[0]
    return json.loads(conn.recv(size).decode())

# =====
# 4. PEER SYNC HANDLER (BOOTSTRAP + REPAIR)
# =====

def handle_peer_sync(conn, db, local_chain, peer_id):
    """
    Handshake → VBlock Sync → Atomic Apply
    """
    try:
        msg = recv_json(conn)
        if not msg or msg.get("type") != "HANDSHAKE":
            return

        expected = compute_handshake_hash(
            local_chain.genesis_h4,
            msg["last_block_number"],
            local_chain.get_block_h4(msg["last_block_number"])
        )

        if expected != msg["handshake_hash"]:
            logging.warning(
                f"HANDSHAKE_FAIL | peer={peer_id} | reason=CHAIN_MISMATCH"
            )
            return

        # Handshake OK
        send_json(conn, {
            "type": "HANDSHAKE_ACK",
            "tip": local_chain.tip
        })

        # Receive VBlocks
        while True:
            data = recv_json(conn)
            if not data or data["type"] == "END_SYNC":
                break

```

```

if data["type"] == "VBLOCK":
    success, err = apply_vblock_atomic(
        db=db,
        sender_id=data["sender"],
        consumed_h4s=data["inputs"],
        new_outputs=data["outputs"],
        new_h1=data["h4"],
        new_seq=data["seq"],
        peer_id=peer_id
    )

    if not success:
        logging.warning(
            f"SYNC_ABORT | peer={peer_id} | block={data['num']} | {err}"
        )
        return

except Exception as e:
    logging.error(f"SYNC_ERROR | peer={peer_id} | {str(e)}")

# =====
# END OF FILE

```